

# **FULL STACK**

## **ASSIGNMENT – 2**

### **IMPLEMENT STUDENT MANAGEMENT SYSTEM USING MONGODB**

**Student Name:** D. Vijaya Sri

**Roll Number:** A24126552262

**Department:** CSM (AI&ML)

**Section:** B

**Subject:** FULL STACK

**Assignment No.:** 2

**Date:** 03-10-2026

**Academic Year:** 2026

**Technology Used:** Node.js with MongoDB

# 1. Objective

To develop a simple student management system using Node.js and MongoDB to store, retrieve, update, delete, search, sort, and manage student information.

# 2. Introduction

MongoDB is a NoSQL database that stores data in flexible document format. In this assignment, a database named **collegeDB** and a collection named **students** are used to manage student records. Node.js is used to connect to MongoDB and perform database operations.

# 3. Operations Implemented

- Create the collegeDB database and students collection
- Insert student records
- Display all students
- Search students by branch, roll number, year, and marks
- Update marks and email
- Delete a student using rollNo
- Sort students by marks
- Create and demonstrate an index on rollNo
- Perform real-time extension queries

# 4. System / Software Requirements

- Node.js
- Visual Studio Code
- MongoDB Server
- MongoDB Compass
- MongoDB Node.js Driver
- Windows and command-line terminal

## 5. Database Setup

The MongoDB Node.js driver was installed using npm.

### Installation Command:

```
npm install mongodb
```

### Connection and Collection Setup:

```
const { MongoClient } = require("mongodb");

const url = "mongodb://127.0.0.1:27017";
const client = new MongoClient(url);

async function main() {
  try {
    await client.connect();

    const db = client.db("collegeDB");
    const students = db.collection("students");

    console.log("Database: collegeDB");
    console.log("Collection: students");
    console.log("Ready to manage student records!");
  } finally {
    await client.close();
  }
}

main();
```

## 6. Insert Student Records

```
const studentData = [
  {
    rollNo: "23CM001",
    name: "Ravi Kumar",
    branch: "CSE-AIML",
    year: 3,
    marks: 85,
    email: "ravi@example.com"
  },
  {
    rollNo: "23CM002",
    name: "Priya Sharma",
    branch: "CSE",
    year: 3,
    marks: 92,
    email: "priya@example.com"
  },
  {
    rollNo: "23CM003",
    name: "Arjun Reddy",
    branch: "ECE",
    year: 2,
    marks: 68,
    email: "arjun@example.com"
  },
  {
    rollNo: "23CM004",
    name: "Sneha Rao",

```

```
        branch: "CSE-AI ML",
        year: 3,
        marks: 78,
        email: "sneha@example.com"
    },
    {
        rollNo: "23CM005",
        name: "Kiran Kumar",
        branch: "MECH",
        year: 2,
        marks: 45,
        email: "kiran@example.com"
    },
    {
        rollNo: "23CM006",
        name: "Anjali Devi",
        branch: "CSE",
        year: 4,
        marks: 88,
        email: "anjali@example.com"
    }
];

const result = await students.insertMany(studentData);
```

### Execution Result:

Students inserted successfully!  
Number of students inserted: 6

## 7. Retrieval Operations

### 7.1 Display All Students

```
const allStudents = await students.find().toArray();

console.log("ALL STUDENTS:");
console.log(allStudents);
```

### 7.2 Display Students Belonging to CSE-AIML

```
const result = await students.find({
  branch: "CSE-AIML"
}).toArray();

console.log("STUDENTS FROM CSE-AIML:");
console.log(result);
```

#### Actual Result:

STUDENTS FROM CSE-AIML:

Ravi Kumar - 23CM001 - 85 marks  
Sneha Rao - 23CM004 - 78 marks

## 7.3 Display Students Scoring More Than 75

```
const result = await students.find({
  marks: { $gt: 75 }
}).toArray();

console.log("STUDENTS WHO SCORED MORE THAN 75:");
console.log(result);
```

### Actual Result:

```
Ravi Kumar - 85 marks
Priya Sharma - 92 marks
Sneha Rao - 78 marks
Anjali Devi - 88 marks
```

## 7.4 Search Student Using rollNo

```
const student = await students.findOne({
  rollNo: "23CM001"
});

console.log("STUDENT FOUND:");
console.log(student);
```

### Actual Result:

```
STUDENT FOUND:

rollNo: 23CM001
name: Ravi Kumar
branch: CSE-AIML
year: 3
marks: 85
email: ravi@example.com
```

## 7.5 Search Students Based on a Condition

```
const result = await students.find({
  year: 3
}).toArray();

console.log("STUDENTS IN YEAR 3:");
console.log(result);
```

### Actual Result:

```
Ravi Kumar - Year 3
Priya Sharma - Year 3
Sneha Rao - Year 3
```

## 8. Update Operations

### 8.1 Update the Marks of a Particular Student

```
const result = await students.updateOne(
  { rollNo: "23CM001" },
  { $set: { marks: 90 } }
);

console.log("Student marks updated!");
console.log("Documents modified:",
  result.modifiedCount);
```

#### Actual Result:

```
Student marks updated!
Documents modified: 1

Ravi Kumar
marks: 90
email: ravi@example.com
```

### 8.2 Update Another Field – Email

```
const result = await students.updateOne(
  { rollNo: "23CM001" },
  { $set: {
    email: "ravi.kumar@example.com"
  } }
);

console.log("Email updated!");
console.log("Documents modified:",
  result.modifiedCount);
```

#### Actual Result:

```
Email updated!
Documents modified: 1

Ravi Kumar
marks: 90
email: ravi.kumar@example.com
```

### 8.3 Delete a Student Using rollNo

```
const result = await students.deleteOne({
  rollNo: "23CM005"
});

console.log("Student deleted!");
console.log("Documents deleted:",
  result.deletedCount);
```

#### Actual Result:

```
Student deleted!
Documents deleted: 1
```

## 9. Sorting Students by Marks

Display students in descending order of marks.

```
const result = await students
    .find()
    .sort({ marks: -1 })
    .toArray();

console.log(
    "STUDENTS SORTED BY MARKS " +
    "(HIGHEST TO LOWEST):"
);
console.log(result);
```

### Actual Result:

STUDENTS SORTED BY MARKS:

1. Priya Sharma - 92 marks
2. Ravi Kumar - 90 marks
3. Anjali Devi - 88 marks
4. Sneha Rao - 78 marks
5. Arjun Reddy - 68 marks

**Explanation:** The value **-1** sorts the marks in descending order. The value **1** would sort them in ascending order.

## 10. Indexing

### 10.1 Create an Index on rollNo

```
const indexName = await students.createIndex({
  rollNo: 1
});

console.log("Index created successfully!");
console.log("Index name:", indexName);
```

#### Actual Result:

```
Index created successfully!
Index name: rollNo_1
```

### 10.2 Display Available Indexes

```
const indexes =
  await students.listIndexes().toArray();

console.log("AVAILABLE INDEXES:");
console.log(indexes);
```

#### Actual Result:

```
AVAILABLE INDEXES:

  _id_
rollNo_1
```

### 10.3 Demonstrate Why Indexing is Useful

```
const result = await students
  .find({ rollNo: "23CM001" })
  .explain("executionStats");

console.log(result.executionStats);
```

#### Important execution details from the actual output:

```
stage: 'IXSCAN'
indexName: 'rollNo_1'
keysExamined: 1
docsExamined: 1
```

This demonstrates that MongoDB used the rollNo index for the search. Indexes help MongoDB locate matching records efficiently, especially as a collection becomes larger.

# 11. Real-Time Extension

## 11.1 Find Students Scoring Above 80

```
const result = await students.find({
  marks: { $gt: 80 }
}).toArray();

console.log(
  "STUDENTS SCORING ABOVE 80:"
);
console.log(result);
```

### Actual Result:

Ravi Kumar - 90 marks  
Priya Sharma - 92 marks  
Anjali Devi - 88 marks

## 11.2 Find Students Scoring Below 50

```
const result = await students.find({
  marks: { $lt: 50 }
}).toArray();

console.log(
  "STUDENTS SCORING BELOW 50:"
);
console.log(result);
```

### Actual Result:

STUDENTS SCORING BELOW 50:  
[]

The result is empty because Kiran Kumar, who originally had 45 marks, was deleted during the required delete operation.

## 11.3 Find the Highest-Scoring Student

```
const result = await students
  .find()
  .sort({ marks: -1 })
  .limit(1)
  .toArray();

console.log("HIGHEST-SCORING STUDENT:");
console.log(result);
```

### Actual Result:

HIGHEST-SCORING STUDENT:  
Priya Sharma - 92 marks

## 11.4 Find Students Belonging to a Particular Branch

```
const result = await students.find({
  branch: "CSE-AIML"
}).toArray();

console.log("CSE-AIML STUDENTS:");
console.log(result);
```

### Actual Result:

```
CSE-AIML STUDENTS:

Ravi Kumar - 90 marks
Sneha Rao - 78 marks
```

## 11.5 Display Students Sorted According to Marks

```
const result = await students
  .find()
  .sort({ marks: -1 })
  .toArray();

result.forEach((student, index) => {
  console.log(
    `${index + 1}. ${student.name} - ` +
    `${student.marks} marks`
  );
});
```

### Actual Result:

```
1. Priya Sharma - 92 marks
2. Ravi Kumar - 90 marks
3. Anjali Devi - 88 marks
4. Sneha Rao - 78 marks
5. Arjun Reddy - 68 marks
```

## 12. Output

The following is a consolidated summary of the actual Node.js execution outputs obtained during the assignment.

```
MongoDB connected successfully!
Database: collegeDB
Collection: students
Ready to manage student records!
```

```
Students inserted successfully!
Number of students inserted: 6
```

```
Student marks updated!
Documents modified: 1
```

```
Email updated!
Documents modified: 1
```

```
Student deleted!
Documents deleted: 1
```

```
Index created successfully!
Index name: rollNo_1
```

```
AVAILABLE INDEXES:
_id_
rollNo_1
```

```
QUERY EXECUTION DETAILS:
stage: 'IXSCAN'
indexName: 'rollNo_1'
keysExamined: 1
docsExamined: 1
```

```
HIGHEST-SCORING STUDENT:
Priya Sharma - 92 marks
```

```
FINAL SORTED LIST:
1. Priya Sharma - 92 marks
2. Ravi Kumar - 90 marks
3. Anjali Devi - 88 marks
4. Sneha Rao - 78 marks
5. Arjun Reddy - 68 marks
```

## 13. Result

The student management system was implemented successfully using Node.js and MongoDB. CRUD operations, conditional queries, sorting, real-time queries, and indexing were executed successfully.

## 14. Conclusion

The assignment demonstrates how MongoDB can be used with Node.js to store and manage student information. The implementation demonstrates retrieval, updates, deletion, sorting, conditional queries, and the use of an index on rollNo for efficient searching.